

LLD Case Study 10: Multi-Car Elevator Control System

Target Patterns: State Pattern, Strategy Pattern, Command Pattern

Key Engineering Focus: Hardware movement states, SCAN / LOOK disk scheduling algorithms, internal/external hall call commands, and multi-elevator coordination.

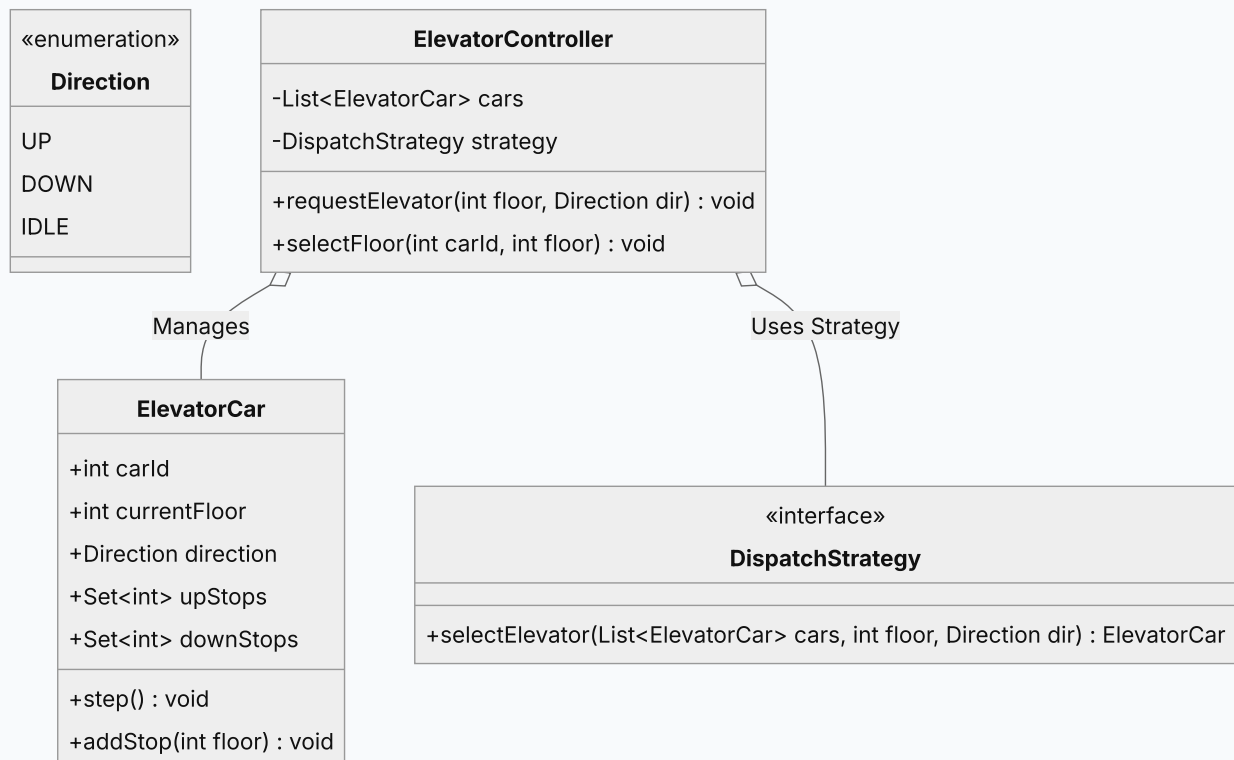
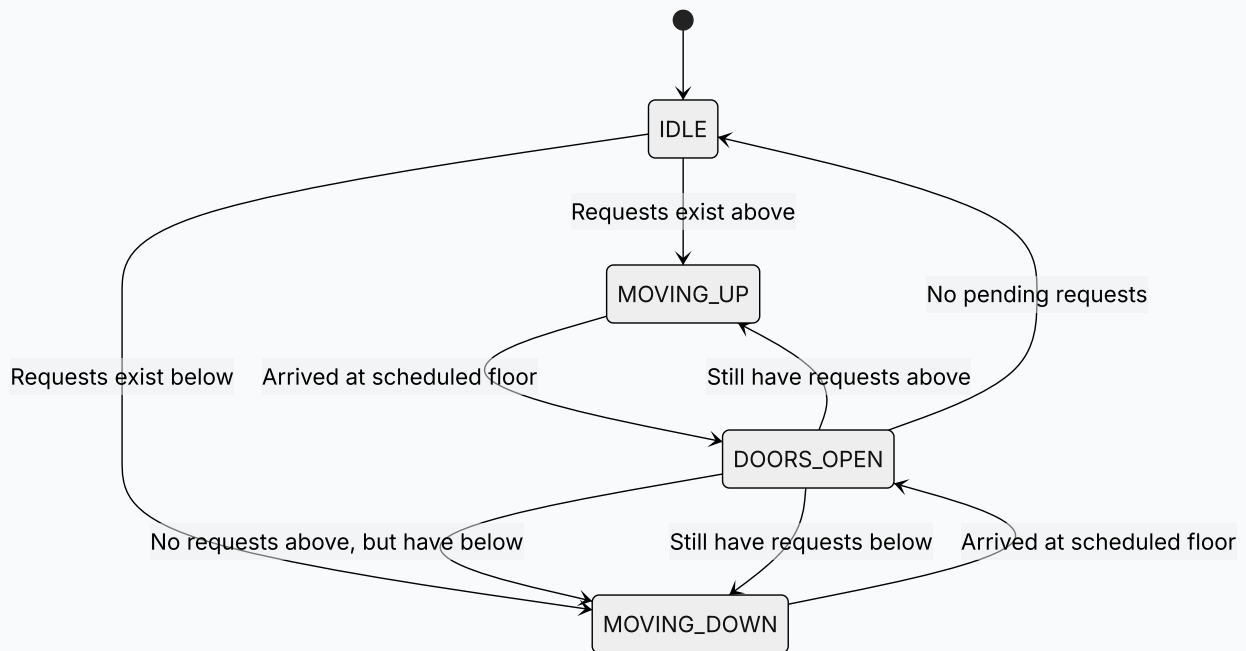
1. Problem Statement & Functional Requirements

Design an automated Elevator Controller governing a bank of M elevators across N floors.

Requirements:

- 1. Movement State Machine (State Pattern):** Elevators transition between `IDLE`, `MOVING_UP`, `MOVING_DOWN`, and `DOORS_OPEN`.
 - 2. Elevator Dispatch Policy (Strategy Pattern):** Route hall calls to the most optimal car using algorithms like **LOOK / SCAN** (continue in current direction servicing all pending stops before reversing).
 - 3. Command Architecture:** Requests come from external hall panels (`direction = UP/DOWN`) and internal car panels (`destination_floor`).
-

2. Architecture & State Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

Behaviour of the LOOK algorithm as implemented

Situation	What the code does	Check
Requests above and below	Finishes all stops in the current direction, then reverses	Tested below
New request behind the car while moving	Goes to the opposite-direction set and is served after the reversal	Tested below
Request for the current floor	Prints "doors opening" and does nothing else	Correct, but no door state is modelled
Idle car after the last stop	<code>direction</code> becomes <code>IDLE</code> and the car stops moving	Tested below
Dispatch tie	<code>min</code> keeps the first car, so car 1 gets every tie	Fine for fairness-insensitive demos; real systems also balance load
Concurrency	Not thread-safe: <code>add_destination</code> and <code>step</code> mutate the same sets	One controller thread, or one lock per car

Tests

This block extends the implementation above. It records door-open events by reading the printed lines, so it also checks the messages the demo prints.

Extensions interviewers ask for

1. **Capacity and weight:** give each car a load; a full car skips hall calls (a high score penalty) but still serves its internal stops.
2. **Door and motion timing:** replace instantaneous `step()` with states (`MOVING` , `DOORS_OPENING` , `DOORS_OPEN` , `DOORS_CLOSING`) driven by a clock; this is the State pattern again and is easiest to test with a fake clock.

3. **Peak traffic modes:** morning rush parks idle cars at the lobby; evening rush spreads them. Add a `DispatchStrategy` per mode and switch by schedule.
4. **Destination dispatch:** riders enter the destination floor at the lobby and the system groups them by direction and floor range; the assignment becomes an optimisation across all pending riders, not a nearest-car rule.
5. **Fault handling:** mark a car out of service, redistribute its pending stops to other cars, and stop assigning it.

Follow-up questions

- *Why LOOK rather than SCAN or FCFS?* FCFS zig-zags and starves far floors. SCAN travels to the end of the shaft even with no requests there. LOOK reverses at the last pending stop, which saves the wasted travel.
- *Can a request starve under LOOK?* Not indefinitely: each reversal clears one direction's set, so a waiting stop is served within two sweeps, assuming the car does not stay permanently busy above it.
- *How would you make the controller thread-safe?* Make the controller the only writer: a queue of commands consumed by one thread that also runs the step clock, which removes locking from the car objects entirely.